

System Configuration & Troubleshooting Guide

Operating System:	Ubuntu Server (Headless, minimal profile)
Networking Engine:	NetworkManager (nmcli / nmtui)
Metrics Standard:	Metric Units Strictly Enforced

1. Production Files & Configurations

The following deployment blueprints outline the architecture of the core infrastructure services containerized via Docker Compose. All operational metrics and telemetry indicators adhere strictly to standard metric sizing rules.

FILE 1: MASTER DEPLOYMENT BLUEPRINT (~/.NAS/DOCKER-COMPOSE.YML)

```
services:
  nas:
    image: filebrowser/filebrowser:latest
    container_name: my_nas
    ports:
      - "8080:80"
    volumes:
      - ~/.nas/data:/srv
      - ~/.nas/config/database.db:/database.db
    restart: unless-stopped

  dashboard:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: my_dashboard
    environment:
      - HOMEPAGE_ALLOWED_HOSTS=*
    ports:
      - "3000:3000"
    volumes:
      - ~/.nas/homepage-config:/app/config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    restart: unless-stopped
```

FILE 2: DASHBOARD INFRASTRUCTURE LINKS (~/.NAS/HOMEPAGE-CONFIG/SERVICES.YAML)

```
- My Infrastructure:
  - Storage Backend:
    icon: filebrowser.png
    href: http://YOUR_WIFI_IP:8080
```

FILE 3: DASHBOARD PARAMETER OVERRIDES (~/.NAS/HOMEPAGE-CONFIG/SETTINGS.YAML)

```
---
# Homepage Settings Configuration
baseurl: http://YOUR_WIFI_IP:3000
disableHostValidation: true
```

Deployment Variable Mapping Note: Replace YOUR_WIFI_IP with the dynamically assigned WLAN address (e.g., 192.168.1.1XX) discovered on your infrastructure layer.

2. Advanced Networking & Expanded DNS Sandbox Matrix

With an operational shift toward local service routing, custom localized domain hijacking, and isolated static site configurations, the deployment layout has been extended with a local DNS Sandbox cluster engine.

Core Architecture Blueprint (Directory Tree Matrix)

The host infrastructure space under the relative home catalog is systematically organized into modular persistence layers:

```
~/nas/
├─ docker-compose.yml      # Global orchestration for Dashboard, Web Engine, and DNS
├─ homepage-config/        # Configuration sets for the visual central matrix dashboard
├─ dns-conf/               # Persistence volume for AdGuard local routing parameters
├─ dns-work/               # Analytical runtime data pool for the DNS sandbox engine
├─ www/                    # Root production directory for raw website assets
└─ index.html              # Target custom lovable HTML source code file
```

FILE 1: UNIFIED SERVICE MATRIX BLUEPRINT (~/.NAS/DOCKER-COMPOSE.YML)

```

services:
  # Service 1: Your Master Dashboard Center
  dashboard:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: my_dashboard
    environment:
      - HOMEPAGE_ALLOWED_HOSTS=*
    ports:
      - "3000:3000"
    volumes:
      - ~/nas/homepage-config:/app/config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    restart: unless-stopped

  # Service 2: Your Custom Lovable Webpage Engine
  my_webpage:
    image: nginx:alpine
    container_name: my_custom_site
    ports:
      - "80:80"
    volumes:
      - ~/nas/www:/usr/share/nginx/html
    restart: unless-stopped

  # Service 3: Your DNS Control Sandbox
  dns_sandbox:
    image: adguard/adguardhome:latest
    container_name: local_dns
    ports:
      - "53:53/udp" # Core DNS Traffic Handling
      - "53:53/tcp"
      - "8080:80/tcp" # Control Dashboard Panel
    volumes:
      - ~/nas/dns-work:/opt/adguardhome/work
      - ~/nas/dns-conf:/opt/adguardhome/conf
    restart: unless-stopped

```

FILE 2: CUSTOM WEBSITE ASSET (~/.NAS/WWW/INDEX.HTML)

```
<!DOCTYPE html><html><head>
  <title>Mathi's Homelab</title>
  <style>
    body { font-family: sans-serif; background: #0f172a; color: #f8fafc; text-align:
center; padding: 50px; }
    h1 { color: #38bdf8; font-size: 3rem; }
    p { color: #94a3b8; font-size: 1.2rem; }
  </style></head><body>
  <h1>Welcome to Mathi's Server Matrix</h1>
  <p>This custom lovable webpage is officially streaming from a headless sandbox server
over local wireless networks.</p></body></html>
```

3. Troubleshooting Ledger & Critical Error Pitfalls

During bare-metal environment execution, several distinct engineering bottlenecks emerged across the networking, storage orchestration, and dashboard interface boundary layers. This structured index establishes operational safeguards for subsequent runs.

Category A: Headless Server Wireless Card Isolation

1. The "Command Not Found" Pitfall

SYMPTOM Running network utility commands (e.g., `iwlist`, `rkill`, `nmcli`) to check or manipulate wireless infrastructure returns a binary execution failure.

ROOT CAUSE Stripped-down production server installations exclude traditional desktop-centric or vintage software packages by default.

RESOLUTION PROTOCOL Route temporary out-of-band management through a physical Ethernet layer and provision the modern network manager ecosystem wrapper:

```
sudo apt update && sudo apt install -y network-manager rkill wpasupplicant
```

2. The "Scanning Not Allowed When Unavailable" Error Loop

SYMPTOM Execution of `nmcli device wifi list` outputs empty layout headers or explicitly errors out with an unavailable block.

ROOT CAUSE 1 (HARDWARE/FIRMWARE LOCK) The physical circuit topology or internal radio card is hard-blocked via an onboard toggle or a system function key combo. Running `rfkill list` outputs `Hard blocked: yes`. Toggling the physical layout switch resolves this condition.

ROOT CAUSE 2 (DAEMON SYSTEM CONFLICT) The default `systemd-networkd` backend rendering agent maintains an exclusive lock on the hardware, preventing `NetworkManager` from successfully driving the radio component.

RESOLUTION PROTOCOL Re-delegate rendering authority directly to `NetworkManager` by adapting the `netplan` definition ruleset located at `/etc/netplan/`:

```
network:
  version: 2
  renderer: NetworkManager
```

Commit the network state change and recycle the management daemons:

```
sudo netplan generate && sudo netplan apply && sudo systemctl restart NetworkManager
```

3. The 802-11-wireless: property missing Profile Creation Glitch

SYMPTOM Standard `nmcli` connection strings abruptly terminate, logging an explicit 802.11 missing property parameter block.

ROOT CAUSE Shell meta-characters, internal spaces, or special symbols inside the SSID target or WPA2 pre-shared key drop breaks down the execution string, stripping downstream parameters from the input parser.

RESOLUTION PROTOCOL Force interactive terminal input parsing targeting the exact device interface, masking raw inputs safely:

```
sudo nmcli --ask device wifi connect "Your_WiFi_Name" ifname wlan0
```

Alternative TUI escape: Execute `sudo nmtui` to initialize the `curses` visual layout and complete the security handshake via text wizard.

Category B: Directory Path & Mapping Architecture Mismatches

1. The Container Ghost Folder Generation Fault

SYMPTOM Service containers fail to fetch local configurations, generating vanilla default instances while dashboard paths drop connection or enter cyclic restart states.

ROOT CAUSE A subtle character or semantic path discrepancy exists between the host file layout and the exact Docker Compose volume definition string (e.g., establishing `~/nas/homepage/config` on disk but binding `~/nas/homepage-config` inside the configuration script). Docker automatically spawns missing paths as empty root-owned directories.

RESOLUTION PROTOCOL Enforce structural validation over SFTP/FileZilla to match a uniform directory model. Rename target endpoints cleanly to: `~/nas/homepage-config/`. Shift configuration assets directly into the root level of the target folder, clearing out arbitrary nested trees.

Category C: Network Layer Security & Browser Header Blocking

1. The ERR_INVALID_HTTP_RESPONSE Break

SYMPTOM Accessing port 8080 yields an explicit invalid HTTP response transmission block in web browsers.

ROOT CAUSE The browser or system container is attempting to negotiate full TLS/SSL validation loops (`https://`) over raw, unencrypted HTTP local infrastructure before a certificate proxy has been bound to the host.

RESOLUTION PROTOCOL Manually force the plain-text schema directly into the address block: `http://`, or drop restrictive context parameters (e.g., `FB_SECURE_COOKIES=true`) from the active container deployment environment variables.

2. The Dashboard Persistent "Host Validation Error" Lockout

SYMPTOM The client browser connects successfully to port 3000, but the application throws a protective white warning blocking layout regarding unverified host headers.

ROOT CAUSE Homepage uses active header validation rules. When the physical machine hops networking layouts or transitions to a separate subnet, the client browser headers conflict with perceived instance boundaries.

RESOLUTION PROTOCOL Pass an environment routing exception catch-all parameter directly inside the dashboard's service configuration block inside the `docker-compose.yml` deployment script:

```
environment:
  - HOMEPAGE_ALLOWED_HOSTS=*
```

Purge and rebuild the service stack completely to ensure absolute state clearance from memory caches:

```
docker compose down && docker compose up -d
```

4. Portability Blueprint (Raspberry Pi Migration)

Because the infrastructure footprint has been decoupled via Docker containers, migrating this exact operational setup to an ARM-based environment (such as a Raspberry Pi) requires no fundamental structural changes. The transition follows a clean, standardized execution lifecycle:

- **Architecture Agnosticism:** Foundational image manifests (including `ghcr.io/gethomepage/homepage`, `nginx:alpine`, and `adguard/adguardhome`) feature cross-platform multi-architecture compilation schemas. Upon stack initialization, Docker automatically evaluates the target hardware arm system and fetches the optimized architecture binary layer.
- **Relative Home Paths:** Utilizing the relative home directory symbol (`~`) allows target paths to seamlessly evaluate from host-specific lines like `/home/ubuntu/nas` to environment-specific variables like `/home/pi/nas` without hardcoding manual execution boundaries.
- **Migration Pipeline:** Provision the standard Docker engine environment on the target Raspberry Pi OS instance. Establish a secure SFTP transport bridge via FileZilla, migrate the full parent directory root structure (`~/nas`) onto the target, and immediately spin up the environment with:

```
docker compose up -d
```

- **Edge Security Reinforcement:** To harden the bare-metal ARM boundary and explicitly safeguard authentication endpoints from persistent brute-force intrusion attacks, deploy automated log parsing/intrusion bans alongside explicit packet filtration profiles.

STEP 1: INSTALL FAIL2BAN INSTANTLY TO AVOID BRUTE-FORCE ATTACKS

```
sudo apt update && sudo apt install -y fail2ban
```

STEP 2: CONFIGURE THE PI'S FIREWALL (UFW) RULES

```
sudo ufw allow 22/tcp  
sudo ufw allow 3000/tcp  
sudo ufw allow 8080/tcp
```

STEP 3: ARM AND ENABLE PACKET FILTRATION

```
sudo ufw enable
```